



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 4 – PRINCIPIOS SOLID



SOLID

S	ingle-Responsibility Principle
O	pen-Closed Principle
L	iskov Substitution Principle
I	nterface Segregation Principle
D	ependency Inversion Principle

SOLID SINGLE-RESPONSIBILITY PRINCIPLE

**When a class violate the
Single responsibility
principle**



SOLID

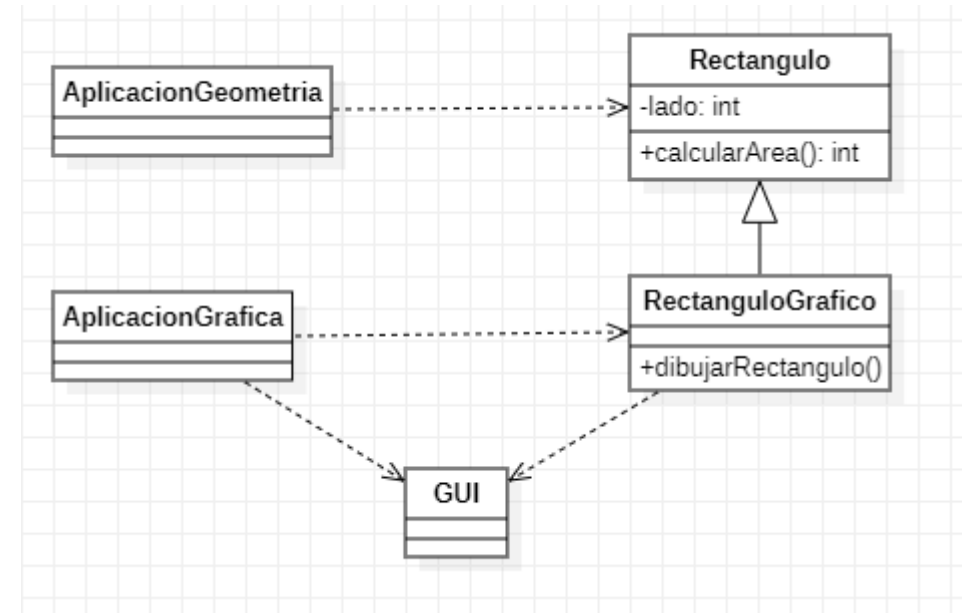
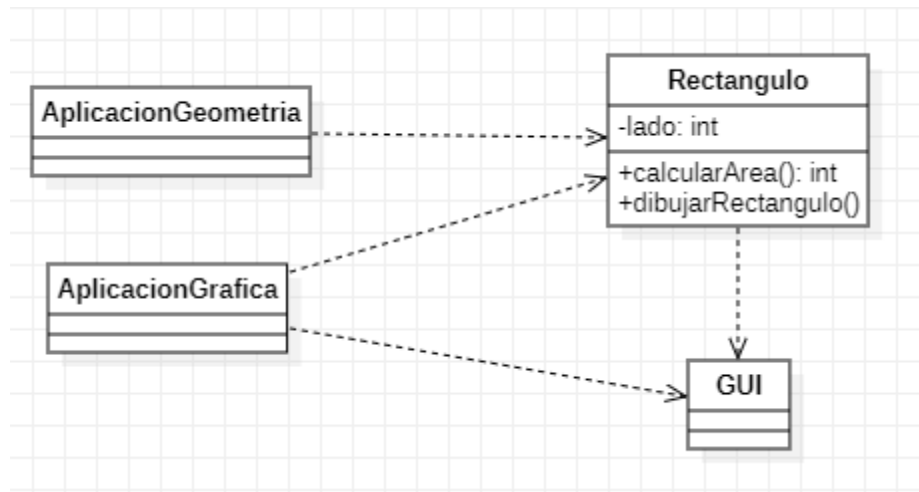
SINGLE-RESPONSIBILITY PRINCIPLE

Cada clase debe tener una única responsabilidad, y esta responsabilidad debe estar contenida únicamente en dicha clase

- Entonces:
 - “Una clase debería tener una única razón por la cual cambiar” (su responsabilidad) *Robert C. Martin, Agile Software Development*
 - Diseñar entidades altamente cohesivas: todo lo que no pertenezca estrictamente a una misma entidad debe separarse
 - Se busca mantener una alta cohesión

SOLID

SINGLE-RESPONSIBILITY PRINCIPLE



SOLID OPEN-CLOSED PRINCIPLE



Open-Closed Principle

Open-chest surgery isn't needed when putting on a coat.

SOLID

OPEN-CLOSED PRINCIPLE

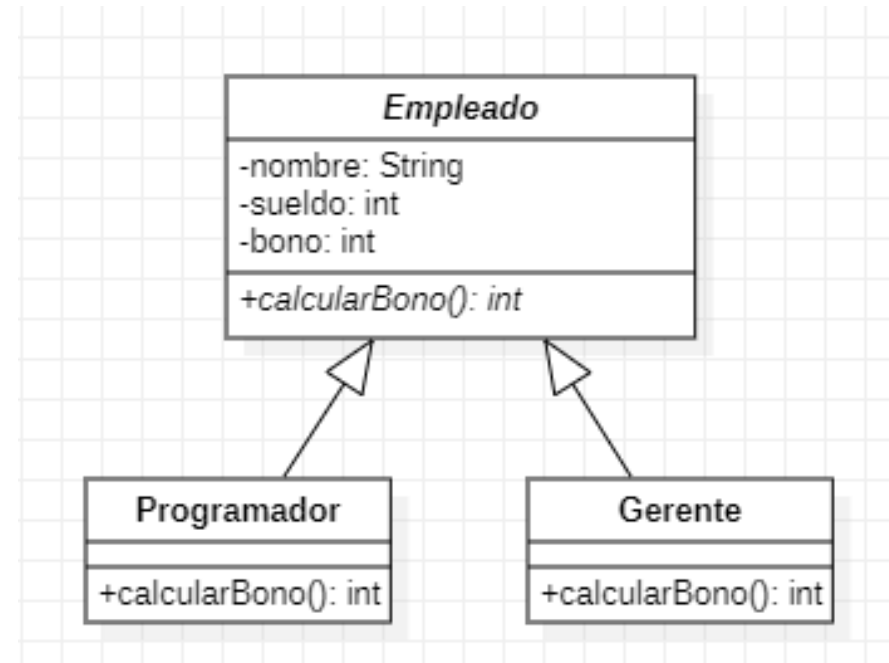
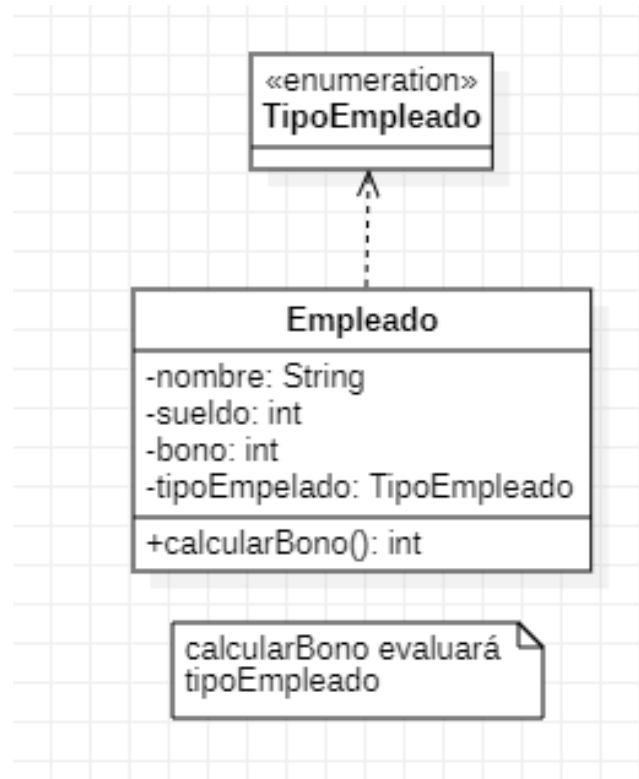
Una entidad debe quedar “abierta” para su extensión, pero “cerrada” para su modificación

Bertrand Meyer, Object-Oriented Software Construction

- Entonces:
 - Deberíamos poder extender el comportamiento de una entidad sin modificar su código fuente. **Herencia**
 - Una entidad es abierta si se puede extender, y cerrada si no se puede modificar
 - Se busca mantener un bajo acoplamiento

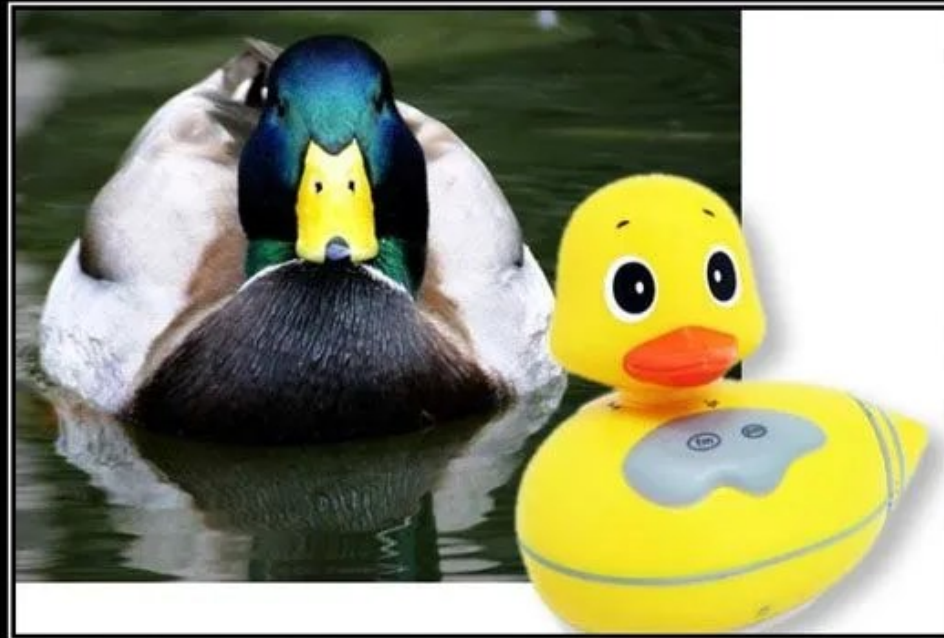
SOLID

OPEN-CLOSED PRINCIPLE



SOLID

LISKOV SUBSTITUTION PRINCIPLE



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You
Probably Have The Wrong Abstraction

SOLID

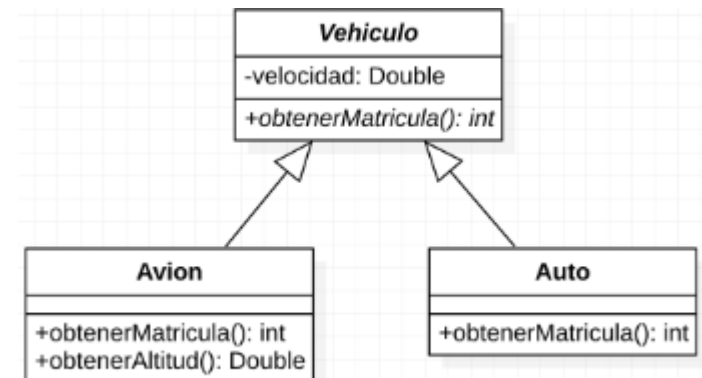
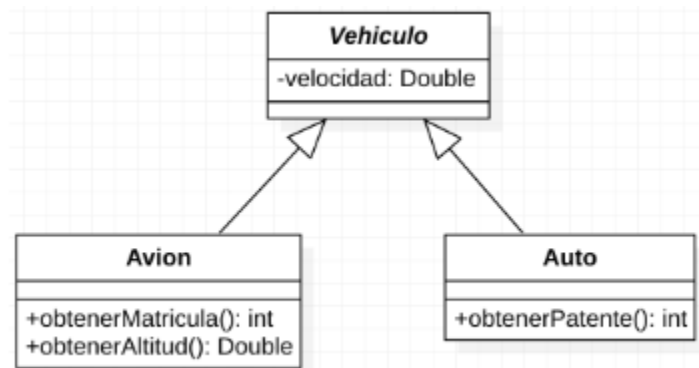
LISKOV SUBSTITUTION PRINCIPLE

Una clase derivada debe poder reemplazarse por cualquier otra que use su clase base sin alterar el correcto funcionamiento de un programa

- Entonces:
 - Si esperamos recibir una clase base en una función, deberíamos poder reemplazarla por cualquier clase derivada
 - Una clase derivada no debe eliminar ni modificar el comportamiento de la clase base
 - Una clase derivada no debe conocer a las demás clases derivadas
 - Si un subtipo tiene un comportamiento que el cliente del supertipo no espera, es una violación al principio

SOLID

LSKOV SUBSTITUTION PRINCIPLE



SOLID INTERFACE SEGREGATION PRINCIPLE



INTERFACE SEGREGATION PRINCIPLE

Don't force the client to depend on things they don't use.

SOLID

INTERFACE SEGREGATION PRINCIPLE

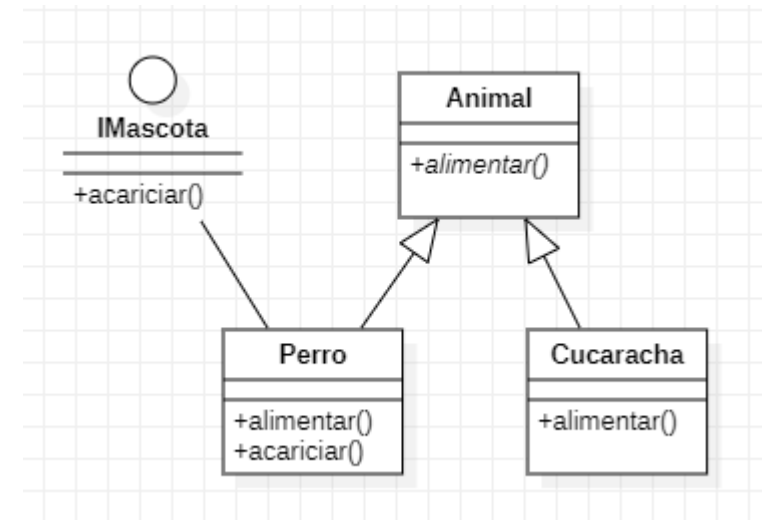
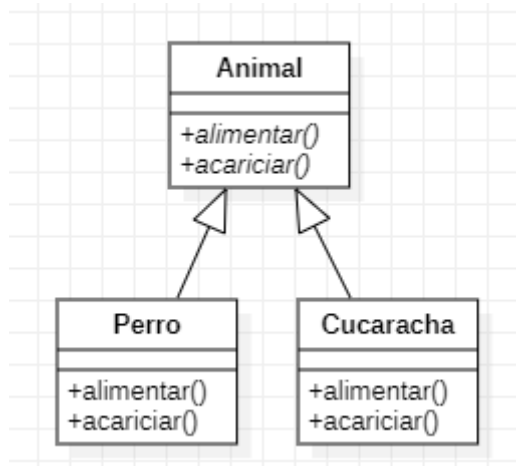
Los clientes no deben estar forzados a depender de métodos que no utilizan

Robert C. Martin, Agile Software Development

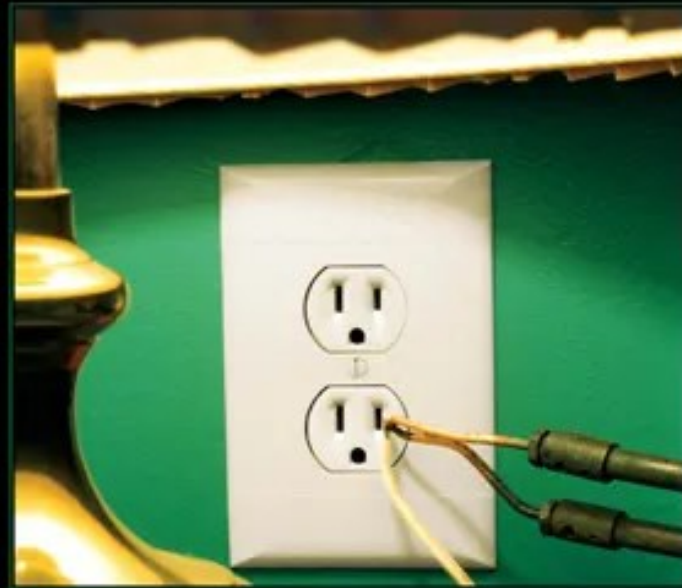
- Entonces:
 - Los clientes solo deben conocer los métodos que van a usar
 - Es mejor tener más interfaces específicas que una única de propósito general

SOLID

INTERFACE SEGREGATION PRINCIPLE



SOLID DEPENDENCY INVERSION PRINCIPLE



DEPENDENCY INVERSION

Would you solder a lamp directly to the electrical wiring in a wall?

SOLID

DEPENDENCY INVERSION PRINCIPLE

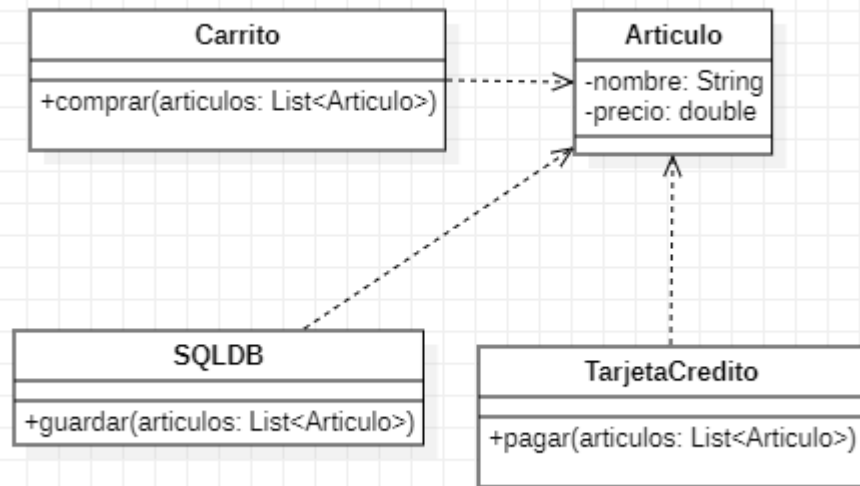
Los sistemas más flexibles son aquellos cuyo código depende de abstracciones, no detalles

Robert C. Martin, Clean Architecture

- Entonces:
 - Los módulos de alto nivel no deberían depender de los de bajo nivel. Ambos deberían depender de abstracciones
 - Las abstracciones no deberían depender de detalles. Los detalles deberían depender de abstracciones
 - Puede implementarse con inyección de dependencias o inversión de control

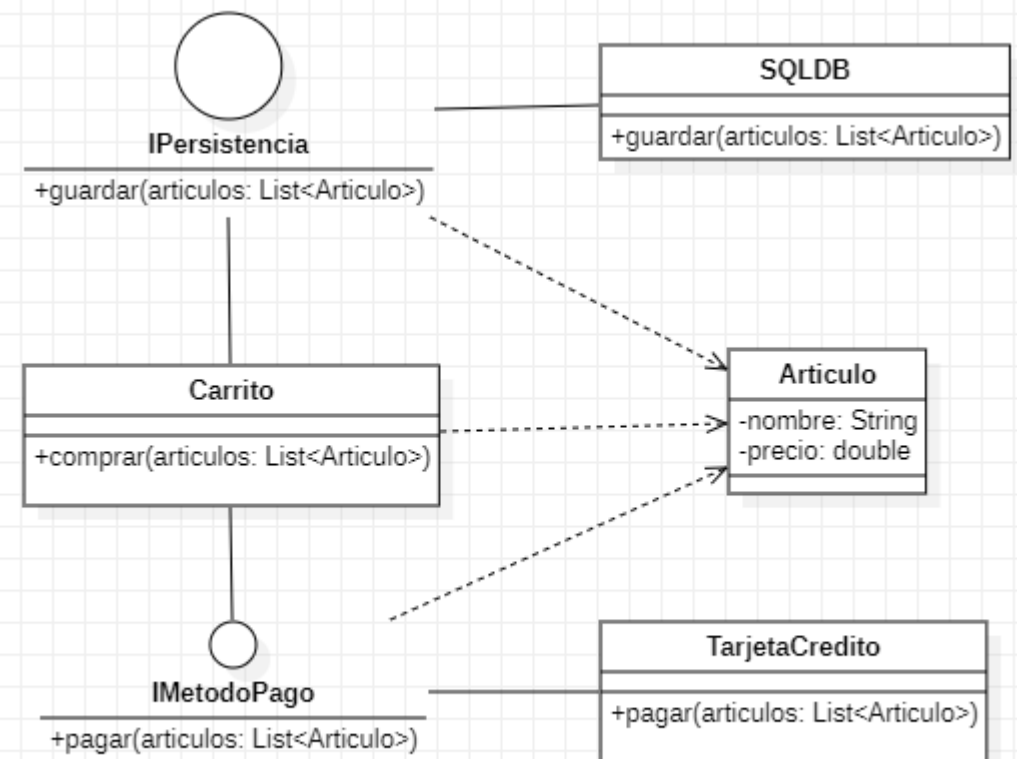
SOLID

DEPENDENCY INVERSION PRINCIPLE



```
public class Compra {
    public void comprar(List<Articulo> articulos) {
        SQLDB db = new SQLDB();
        db.guardar(articulos);

        TarjetaCredito tarjetaCredito = new TarjetaCredito();
        tarjetaCredito.pagar(articulos);
    }
}
```



SOLID RESUMEN

- Utilizar los principios para reducir el acoplamiento y facilitar el cambio
 - Separar responsabilidades con SRP para aislar cambios
 - Diseñar según OCP para simplificar las adiciones/extensiones
 - Respetar el LSP cuando se utilizan abstracciones
 - Minimizar la dependencia de interfaces a través de ISP
 - Introducir abstracciones para dirigir las dependencias (DIP)

BUENAS PRÁCTICAS

- Seguir convención de nombrado de variables, clases y métodos
- Comentarios claros y concisos
- Indentación
- Portabilidad
- Reutilizamiento y escalado
- Pruebas exhaustivas

BIBLIOGRAFÍA

- The SOLID Principles. Klaus Iglberger, Core C++, Mayo 2020
- Martin, R. C. (2003). Agile software development: principles, patterns, and practices. Prentice Hall PTR. Capítulos 8, 9, 10, 11 y 12



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 2 – POO Y DIAGRAMAS DE CLASE

¿Qué abordaremos en la próxima clase?